

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



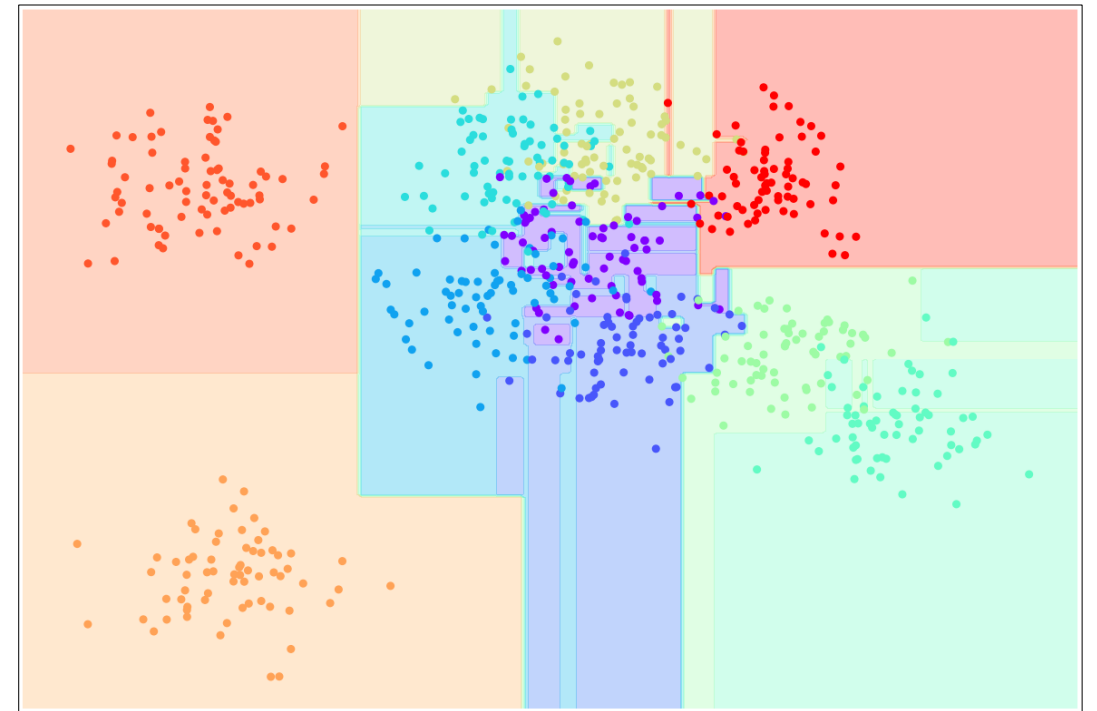
IIC2613 - Inteligencia Artificial

Ensamblés

Denis Parra
Dpto. Ciencia de la Computación

Árboles pueden sufrir de serios problemas de sobreajuste, ¿de dónde viene este problema?

- Muestra es progresivamente reducida al crear nuevos nodos en el entrenamiento (al bajar en el árbol).
- Si hay muchos atributos, existe alta probabilidad de elegir alguno bueno en los datos de entrenamiento, pero “inútil” en cuanto a generalización.

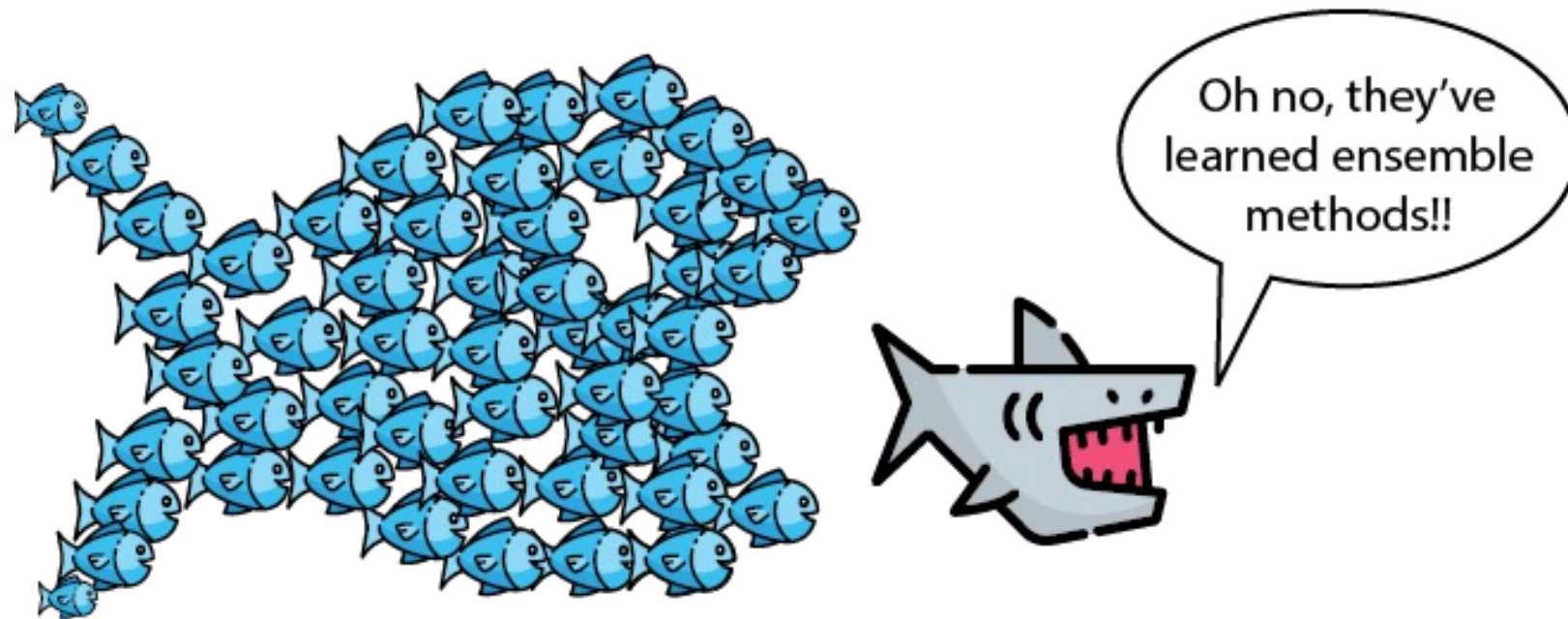


Busquemos un enfoque distinto para enfrentar el sobreajuste

- Consideremos un escenario donde queremos estimar la probabilidad de ocurrencia de un evento.
- En ausencia de un experto, ¿cuál es la mejor manera de realizar la estimación?
- Una encuesta permite estimar la probabilidad de un evento a partir de un número “limitado” de “mediciones”
- La muestra y la metodología juegan un rol fundamental para el éxito de la estimación.

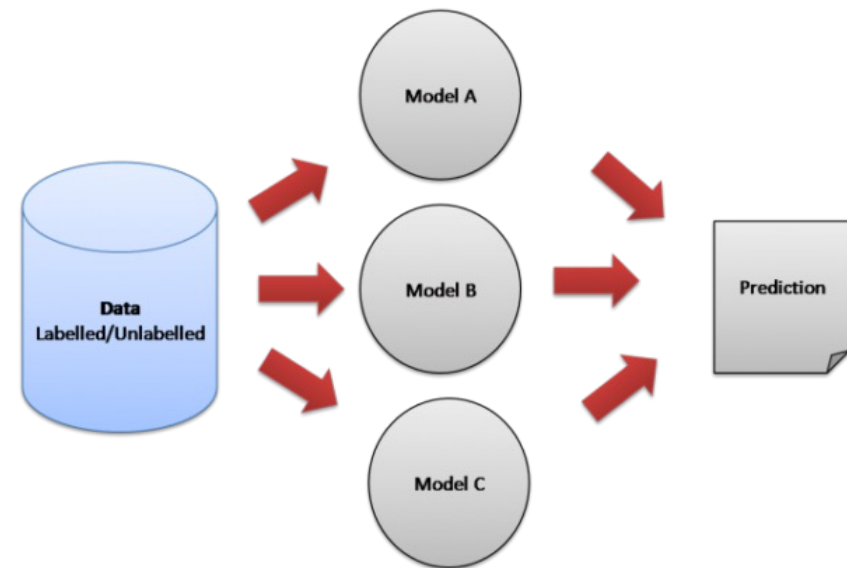


En Machine Learning, estas ideas se formalizan en los métodos de ensamble, los cuales buscan combinar modelos para generar uno **mejor**



En Machine Learning, estas ideas se formalizan en los métodos de ensamble, los cuales buscan combinar modelos para generar uno **mejor**

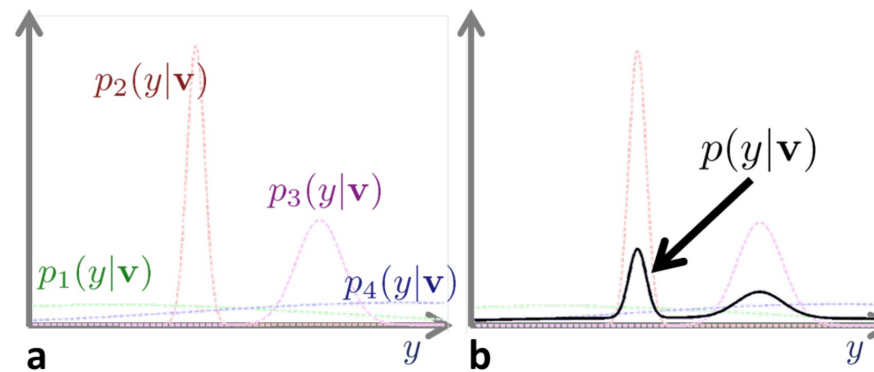
- Desde una perspectiva de **ensambles**, si queremos estar seguros de algo, le podríamos preguntar a más de un modelo.
- Así, si le preguntamos a suficientes, es probable que la moda o el promedio de las predicciones sean cercanas a la realidad.
- **¿Basta entonces con entrenar muchos modelos para eliminar el sobreajuste?**



¿Qué condición deben cumplir entonces los modelos para que esta idea funcione?

Una opción son los ensambles de modelos con **baja correlación**

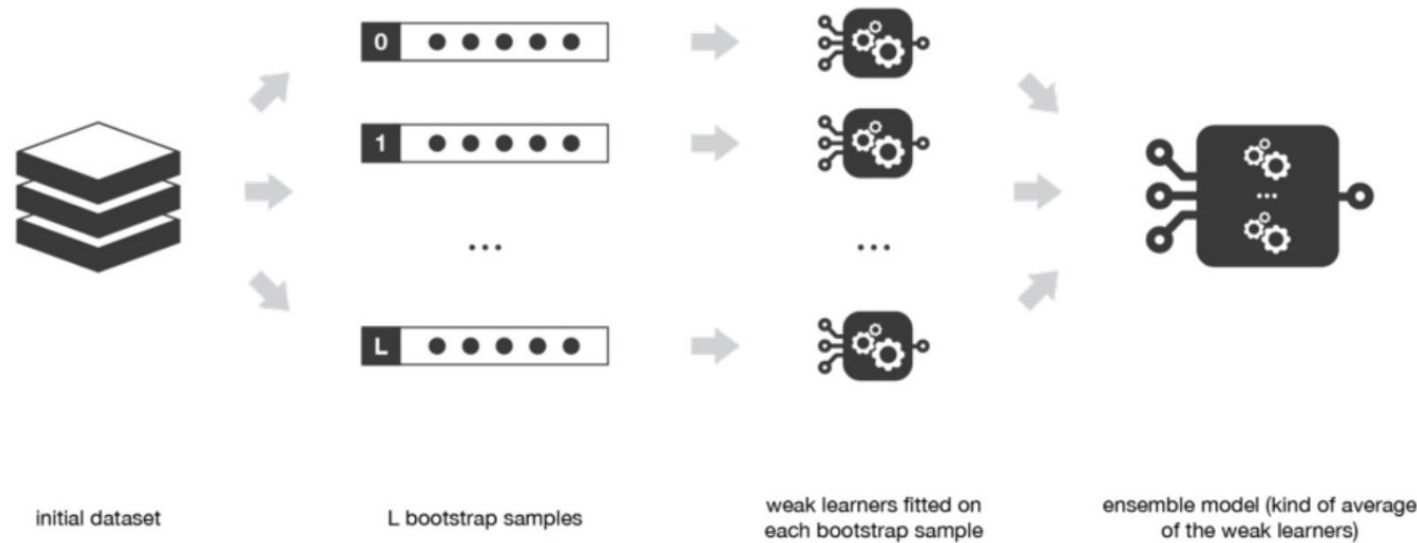
- Si el **patrón de error** (acierto) es **distinto** para todos, es altamente probable que, en promedio, la respuesta del ensamble sea correcta (**errores se cancelan**).
- Para que esto ocurra, todos los **modelos** del ensamble deben ser **débiles**, es decir, ser un poco mejores que dejar la decisión al azar.



¿Cómo logramos esto con los árboles?

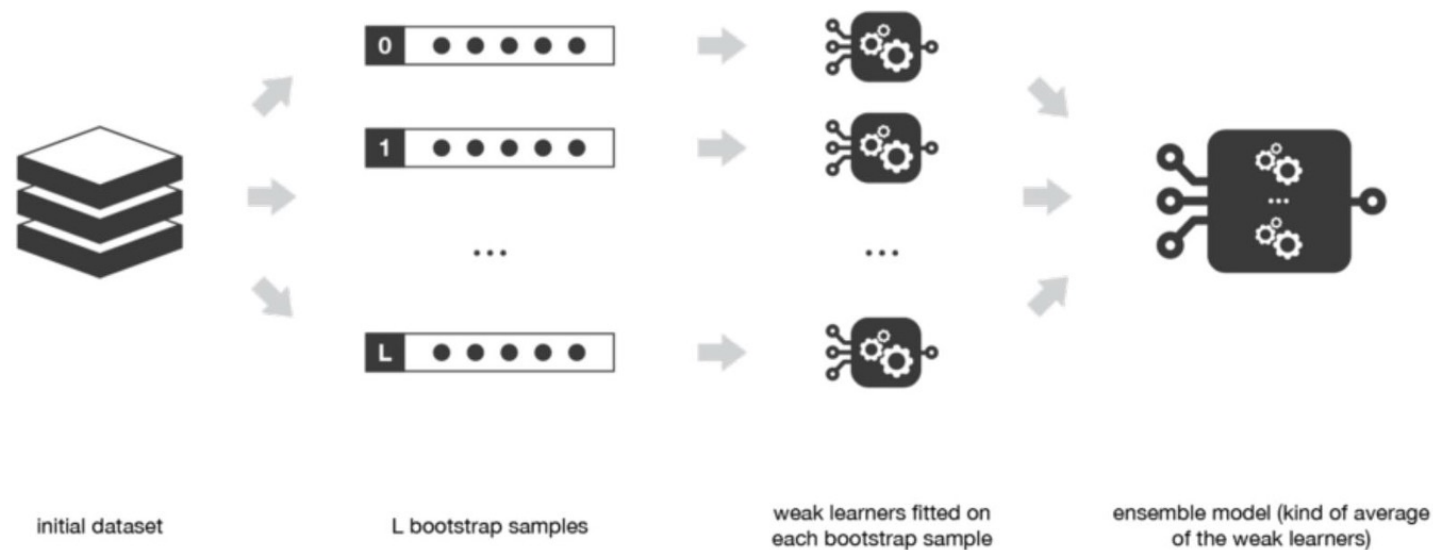
Bagging: La solución sigue con las muestras aleatorias de datos

- Si cada modelo ve sólo una parte de los datos (elegida aleatoriamente), es probable que la correlación entre ellos disminuya.
- **Bagging** (Bootstrap aggregating) permite reducir la varianza del modelo para evitar sobreajuste



Bagging: La solución sigue con las muestras aleatorias de datos

- Si cada modelo ve sólo una parte de los datos (elegida aleatoriamente), es probable que la correlación entre ellos disminuya.
- **Bagging** permite reducir la varianza del modelo para evitar sobreajuste (overfitting)

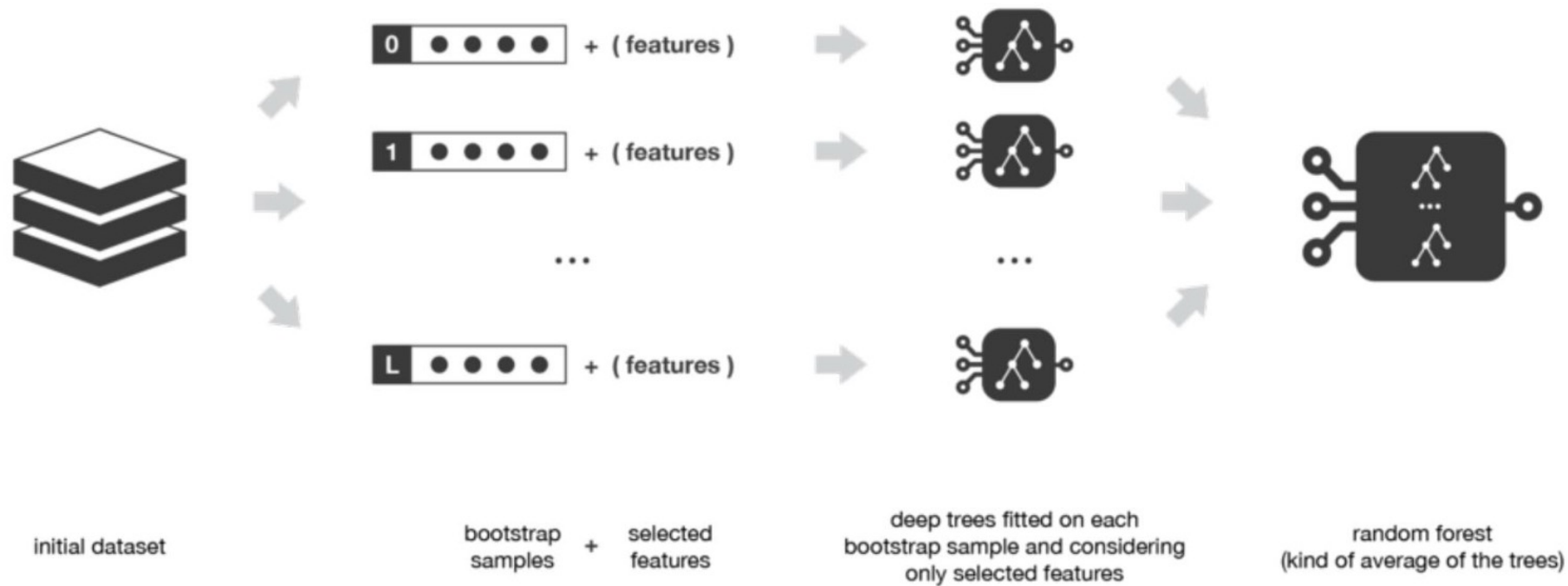


Pero ...

¿Qué pasa si hay una variable muy buena?

La solución está nuevamente en las **muestras aleatorias**, pero ahora de **atributos**

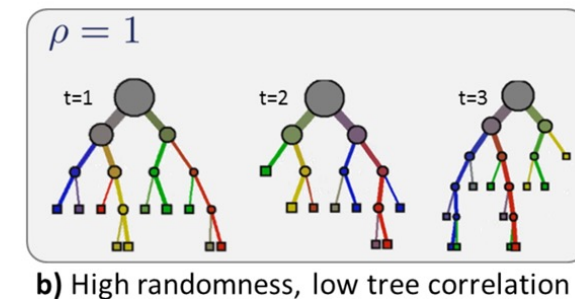
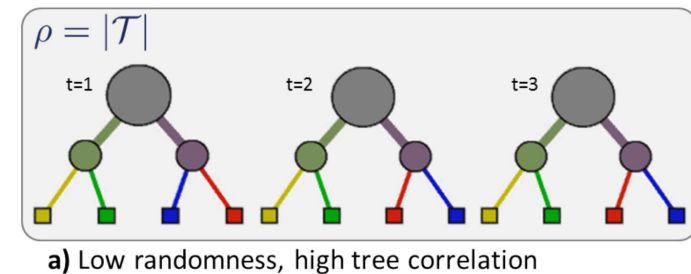
Bagging – Random Forests



Fuente: <https://towardsdatascience.com/ensemble-methods-bagging-boosting-and-stacking-c9214a10a205>

La solución está nuevamente en las **muestras aleatorias**, pero ahora de **atributos**

- Cada vez que se hace un **split**, se elige un **subconjunto aleatorio de los atributos**.
- Luego, para cada uno de ellos, se calcula la ganancia de información.
- Otra ventaja de esto es que se limita la profundidad del árbol, reduciendo la complejidad de este.
- Por lo general, se utiliza una muestra proporcional a la **raíz del número de atributos**.



La introducción de todas estas estrategias “aleatorizantes”, transforma a los árboles en *Random Forests*

Algorithm 15.1 *Random Forest for Regression or Classification.*

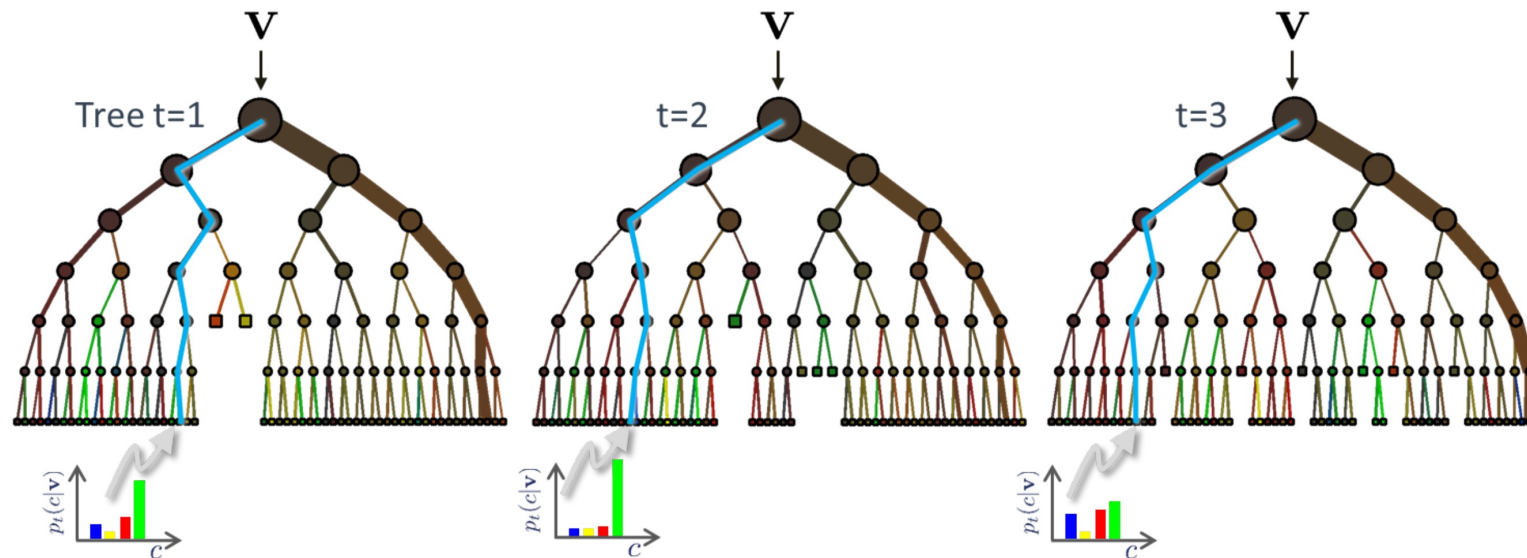
1. For $b = 1$ to B :
 - (a) Draw a bootstrap sample $\mathbf{Z}^*$ of size N from the training data.
 - (b) Grow a random-forest tree T_b to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size n_{min} is reached.
 - i. Select m variables at random from the p variables.
 - ii. Pick the best variable/split-point among the m .
 - iii. Split the node into two daughter nodes.
2. Output the ensemble of trees $\{T_b\}_1^B$.

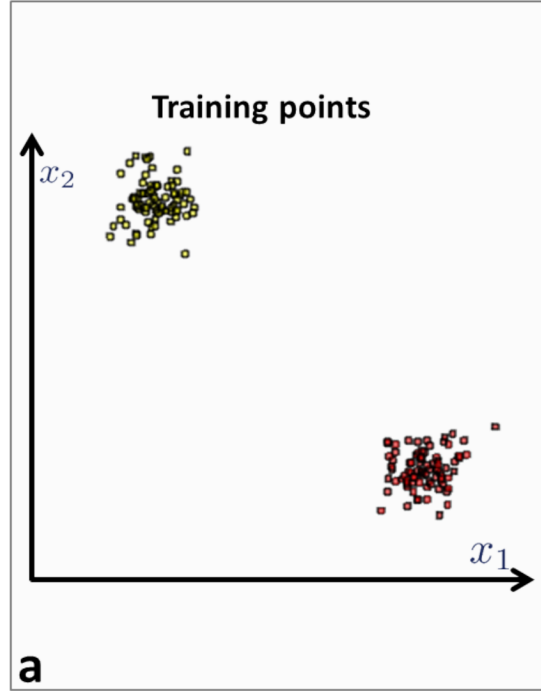
To make a prediction at a new point x :

Regression: $\hat{f}_{\text{rf}}^B(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$.

Classification: Let $\hat{C}_b(x)$ be the class prediction of the b th random-forest tree. Then $\hat{C}_{\text{rf}}^B(x) = \text{majority vote } \{\hat{C}_b(x)\}_1^B$.

La introducción de todas estas estrategias “aleatorizantes”,
transforma a los árboles en *Random Forests*





a

Forest training

Learnt decision boundaries

Learnt trees

Tree 43

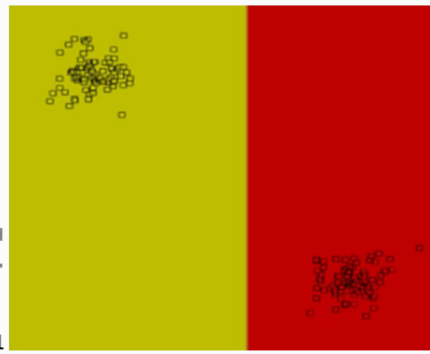
Tree 121

b

Forest testing

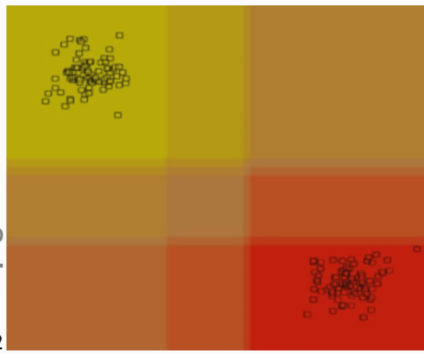
$T=1$

c_1



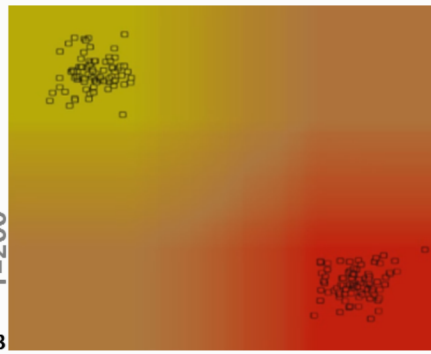
$T=8$

c_2



$T=200$

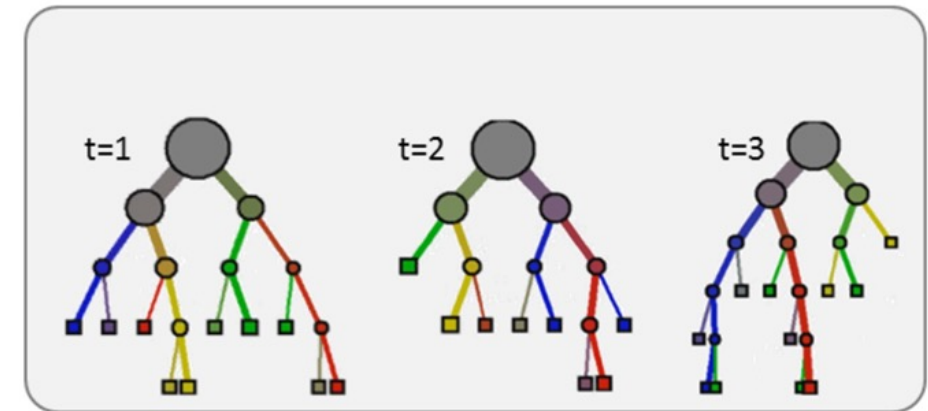
c_3



c

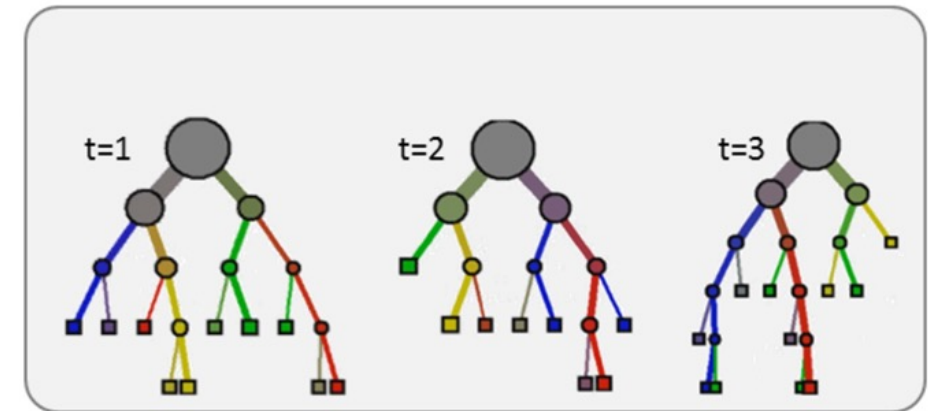
Algunas consideraciones relacionadas con la evaluación

- ¿Cómo podemos cuantificar el poder de generalización de un Random Forest, utilizando únicamente el conjunto de entrenamiento?
- ¿Cómo podemos identificar ahora las variables más relevantes?



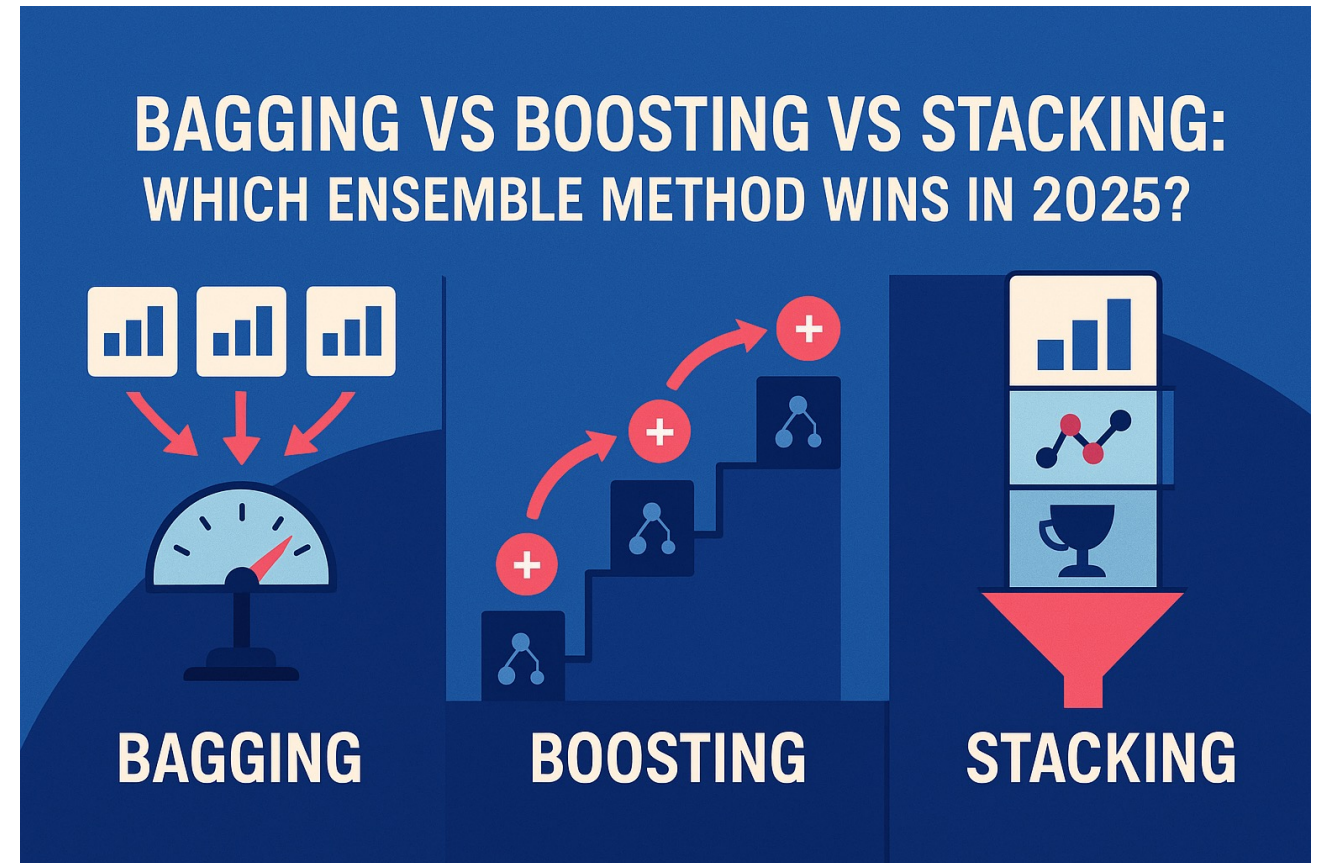
Algunas consideraciones relacionadas con la evaluación

- Es posible cuantificar la generalización directamente a partir del set de entrenamiento, usando ejemplos no vistos por los árboles (*out-of-bag error*).
- Para medir la **importancia** de una variable, basta con **permutar su valor entre los ejemplos** (¿por qué?).



Random Forests es solo uno de muchos tipos posibles de ensamble

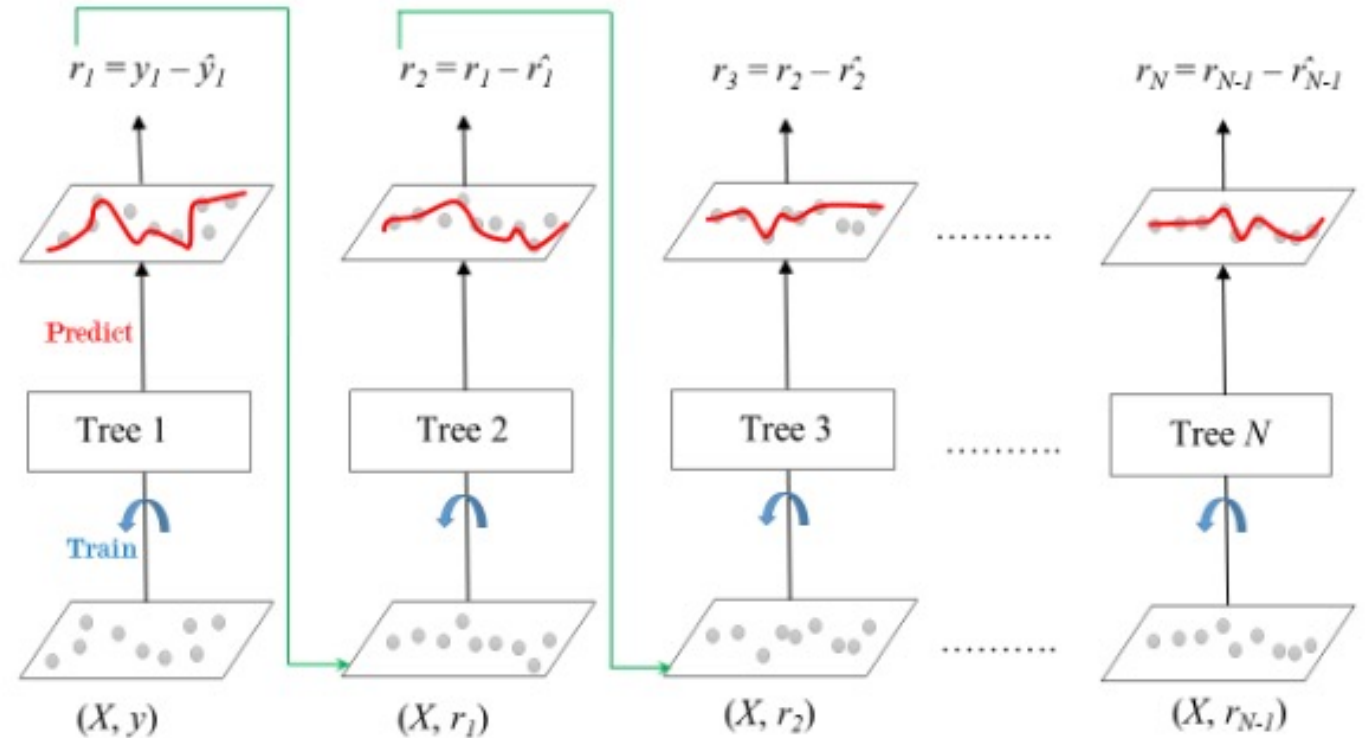
- Random Forests utilizan una estrategia llamada *bagging*, que busca ensamblar múltiples modelos, donde la baja correlación entre ellos permite mejorar el rendimiento.
- Una *manera distinta* de hacer ensambles es intentar que exista una correlación explícita entre los modelos, pero enfocada en la reducción del error: *boosting*. Estos son modelos que predicen de forma secuencial.
- Si el ensamble (combinación) de estos modelos se hace utilizando otro clasificador, entonces el esquema se conoce como *stacking*.

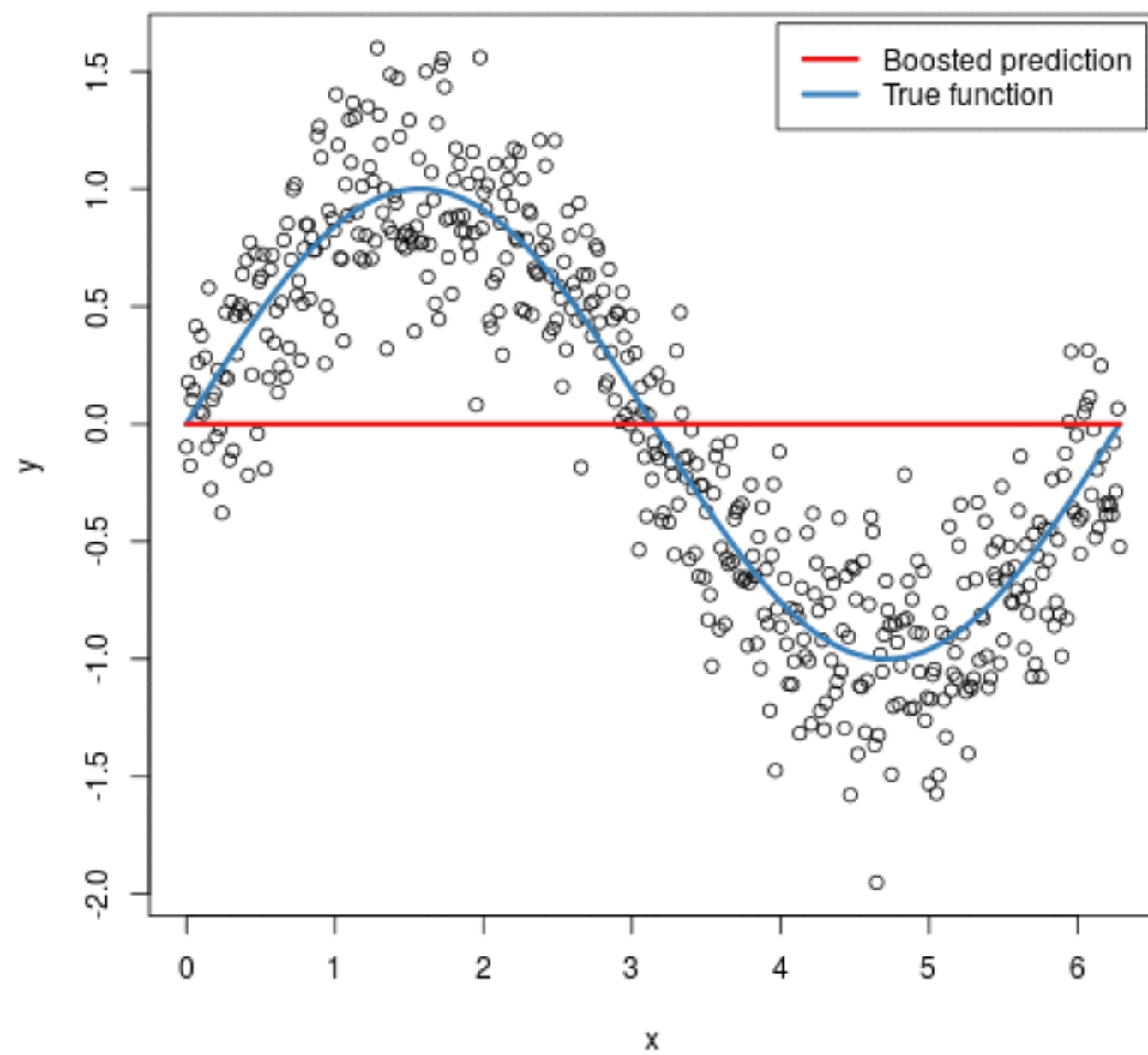


Fuente: <https://machinelearningmastery.com/bagging-vs-boosting-vs-stacking-which-ensemble-method-wins-in-2025/>

Boosting es una estrategia que intenta reducir sesgo de manera secuencial

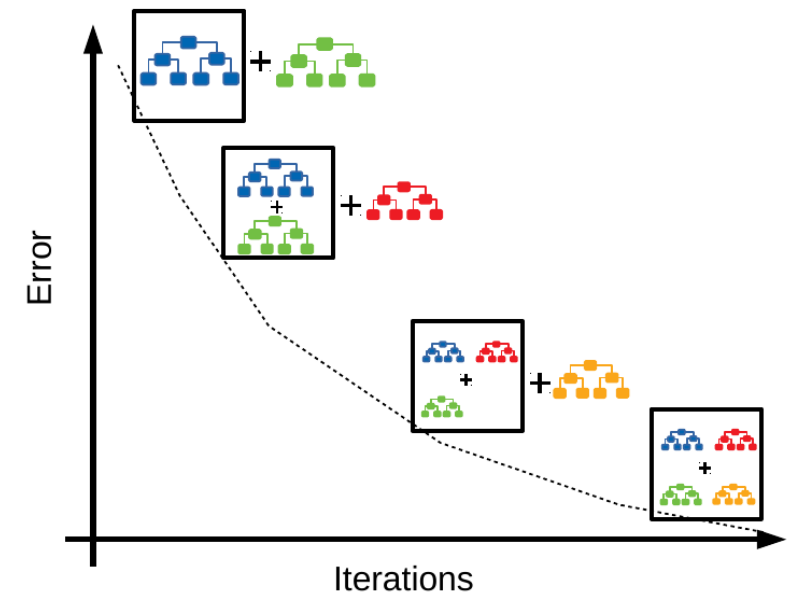
- El primer modelo intenta predecir la variable objeto y se produce un error o residual
- El segundo modelo intenta predecir el residuo del modelo anterior, y queda un nuevo residuo
- Los siguientes modelos de forma secuencial van prediciendo el error





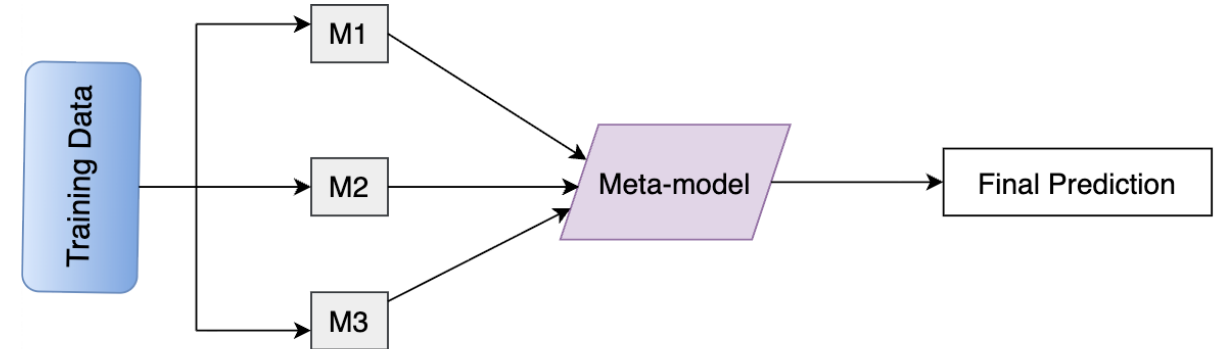
¿Cómo es el rendimiento de *Gradient Boosting*?

- En la práctica, el proceso anterior funciona muy bien (con ciertos agregados).
- Variedades modernas, como **XGBoost** o **LightGBM**, o **CATBoost** son ideales para procesar grandes volúmenes de **datos tabulares**, con rendimiento muchas veces superior a las redes neuronales profundas tradicionales.

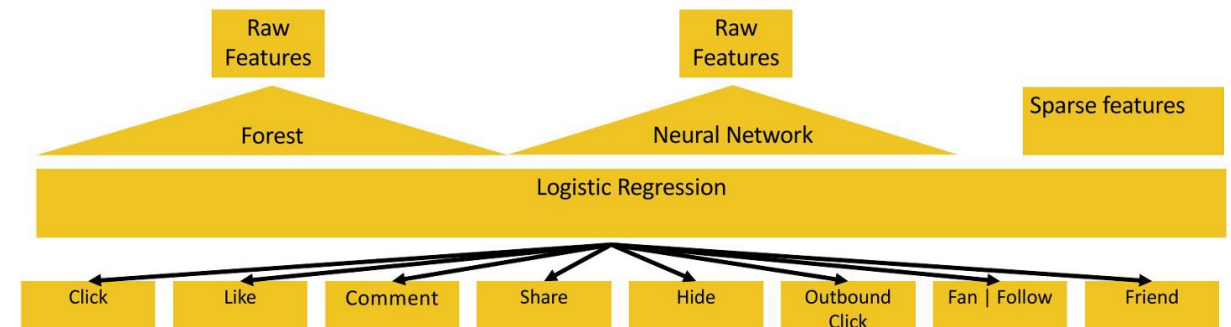


Stacking consiste en entrenar un metamodelo

- En la técnica de **stacking**, entrenamos un meta-modelo que toma como entrada las salidas de otros modelos previamente entrenados
- Esta técnica también suele ser usada porque permite hacer inferencia de forma liviana (con menos features de entrada), ideal para sistema en producción como **Facebook en 2017 (LSRS workshop)**

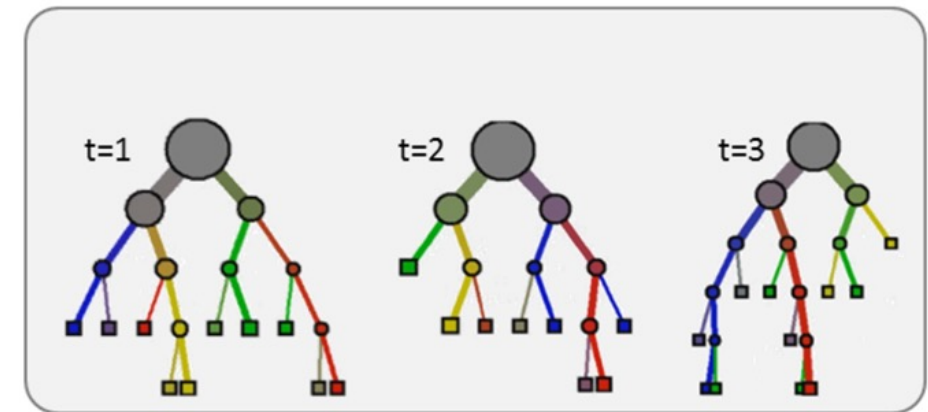


- Train Neural nets to predict events
 - Discard final layer, use **final layer outputs** as features
- Add sparse features such as text or content ID



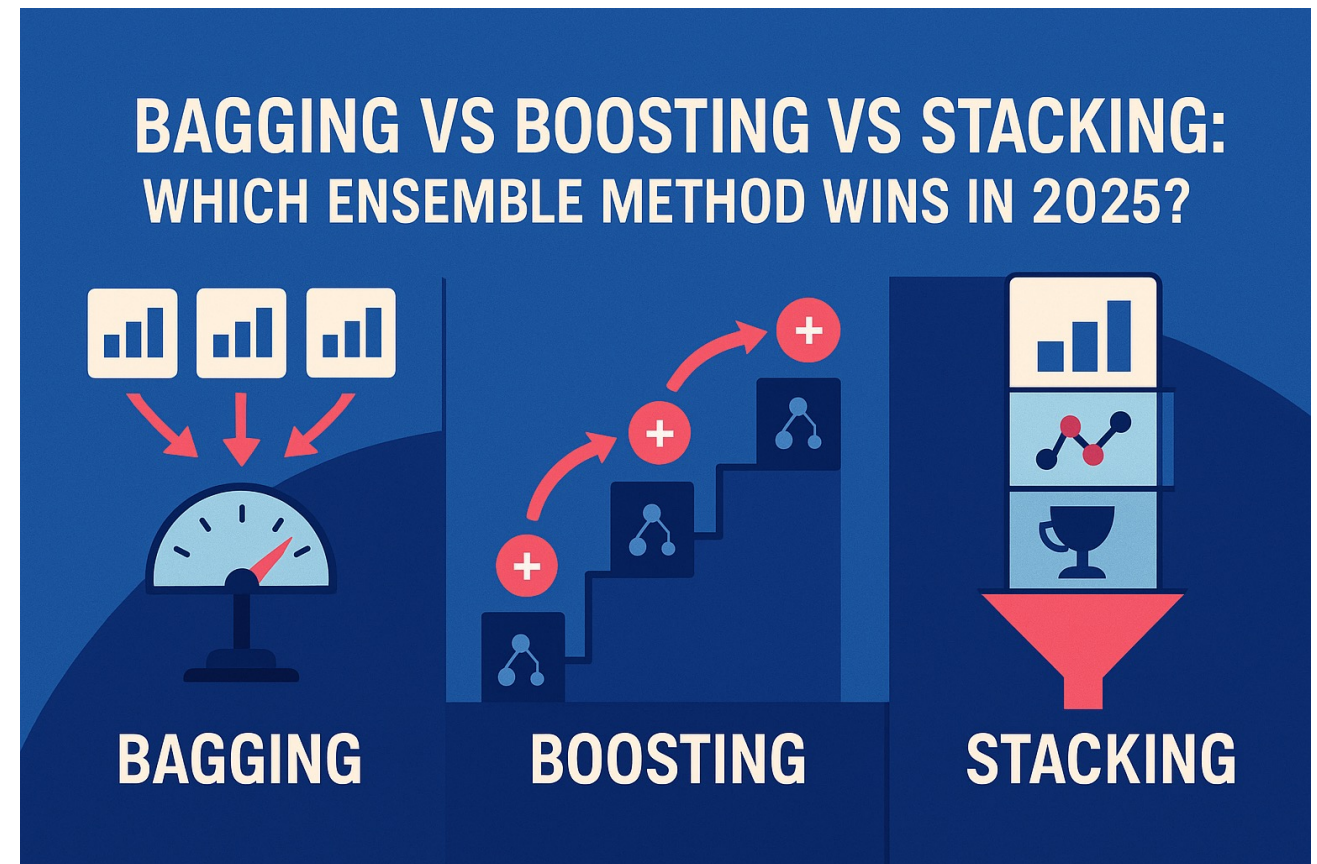
Ensamblajes son ampliamente utilizados en la práctica

- Ensamblajes de árboles de decisión son, al igual que los árboles de decisión, fácilmente interpretables.
- Los ensamblajes se pueden hacer combinando diferentes tipos de modelos (no sólo árboles de decisión)
- Rendimiento es altamente competitivo en problemas con datos *tabulares*.
- Gracias a aleatorización en su construcción, son altamente *resistentes* al *sobreajuste*.



Resumen de puntos clave en esta clase

- Random Forests utilizan una estrategia llamada *bagging*, que busca ensamblar múltiples modelos, donde la baja correlación entre ellos permite mejorar el rendimiento reduciendo la varianza
- Una *manera distinta* de hacer ensambles es intentar que exista una correlación explícita entre los modelos, pero enfocada en la reducción del error: *boosting*. Estos son modelos que predicen de forma secuencial.
- Si el ensamble (combinación) de estos modelos se hace utilizando otro metamodelo adicional, entonces el esquema se conoce como *stacking*.



Fuente: <https://machinelearningmastery.com/bagging-vs-boosting-vs-stacking-which-ensemble-method-wins-in-2025/>

Ejemplos de código disponibles en Canvas

- Clasificación de dígitos con kNN
- Árboles de decisión desde cero
- Regresión con árboles de decisión
- Sobreajuste y validación cruzada en árboles de decisión y ensambles
- Uso de ensambles con datos reales



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



IIC2613 - Inteligencia Artificial

Ensamblés

Denis Parra
Dpto. Ciencia de la Computación